TP — Architecture hexagonale avec Spring Boot 4+

A rendre dans un repo git

Objectif

Construire, à partir d'un dossier vide, une petite API REST de catalogue de cafés organisée selon l'**architecture hexagonale** (aussi appelée *ports et adaptateurs*), avec les pratiques actuelles de Spring Boot 4.1.1. Le métier reste volontairement simple pour garder l'attention sur l'organisation du code : où ranger chaque classe, et dans quel sens vont les dépendances.

Aucune connaissance préalable de l'architecture hexagonale n'est nécessaire. Il faut connaître les bases de Java (classes, interfaces, records) et savoir utiliser un terminal.

Élément	Valeur
Projet	`coffee-hexa-simple`
Group	`org.formation`
Package racine	`org.formation.coffee`
Build	Maven (via le Maven Wrapper `mvnw`)
Java	25
Spring Boot	4.1.1

Les cinq idées à retenir

1. Le **domaine** est au centre : du Java pur, sans Spring ni JPA, qui porte les règles métier.
2. L'**application** déclare des **ports** sous forme d'interfaces : les ports entrants décrivent ce qu'elle offre, les ports sortants ce dont elle a besoin.
3. Les **adaptateurs** traduisent, rien de plus : JSON ↔ commandes, entités JPA ↔ domaine, exceptions ↔ codes HTTP.
4. **Toutes les dépendances pointent vers l'intérieur** : le domaine n'importe rien de technique, l'application n'importe aucun adaptateur.
5. Une **seule classe de configuration** branche le cœur en Java pur sur Spring.

Ce que vous allez construire

Une API REST qui offre quatre opérations :

Appel HTTP	Rôle
`GET /api/coffees`	Lister tous les cafés avec leurs avis
`GET /api/coffees/{id}`	Obtenir un café avec ses avis
`POST /api/coffees`	Créer un café
`PATCH /api/coffees/{id}`	Modifier le nom et/ou le prix d'un café

Les règles métier :

- Le nom d'un café est obligatoire et fait au plus 255 caractères.
- Un prix est compris entre 0 et 200.
- La note d'un avis est comprise entre 1 et 5.

Les erreurs sont renvoyées au format JSON standard **problem details** (RFC 9457) :

- 404 quand un café n'existe pas ;
- 422 quand une règle métier est violée ;
- 400 quand la requête elle-même est mal formée.

Les technologies utilisées :

Outil	Version	Rôle
Java	25	Langage et environnement d'exécution
Spring Boot	4.1.1	Framework applicatif
Spring Data JPA / Hibernate	gérée par Boot	Accès à la base de données
Flyway	gérée par Boot	Migrations du schéma de base
H2	gérée par Boot	Base en mémoire pour le développement
MapStruct	1.6.3	Génère le code de conversion entre couches
JSpecify	gérée par Boot	Annotations indiquant ce qui peut être `null`
springdoc-openapi	3.1.0	Swagger UI pour essayer l'API

Étape 0 — Installer les prérequis

1. **Un JDK 25**, par exemple Temurin 25 depuis Adoptium (https://adoptium.net). Vérifier l'installation :

```
java -version
# openjdk version "25..."
```

2. **Un IDE** : IntelliJ IDEA (l'édition Community suffit), VS Code avec les extensions Java, ou Eclipse.
3. **curl**, pour appeler l'API depuis le terminal. Il est déjà présent sous macOS, Linux et Windows 10+.

Inutile d'installer Maven : le projet embarque le *Maven Wrapper* (`mvnw`), qui télécharge lui-même la bonne version.

> **Sous Windows :** partout où ce TP indique `./mvnw`, tapez `mvnw.cmd`.

Étape 1 — Créer le projet et les dépendances

1.1 Générer le squelette avec Spring Initializr

Ouvrir https://start.spring.io (ou l'assistant *New Project* → *Spring Boot* de l'IDE) et remplir le formulaire :

Champ	Valeur
Project	**Maven**

Champ	Valeur
Language	**Java**
Spring Boot	**4.1.1** (si elle n'est pas proposée, prendre la 4.1.x la plus récente, puis mettre la version du parent à 4.1.1 dans le `pom.xml`)
Group	`org.formation`
Artifact	`coffee-hexa-simple`
Name	`coffee-hexa-simple`
Package name	`org.formation.coffee`
Packaging	Jar
Java	**25**

> **Attention au package :** par défaut, Initializr propose `org.formation.coffee-hexa-simple`. Remplacez-le par `org.formation.coffee`.

Cliquer sur **ADD DEPENDENCIES** et sélectionner les starters suivants :

Starter à cocher	Catégorie Initializr	Rôle dans le projet
Spring Web	Web	Adaptateur entrant : l'API REST
Validation	I/O	Vérification de la forme des requêtes (`@Valid`, `@NotBlank`)
Spring Data JPA	SQL	Adaptateur sortant : accès à la base
Flyway Migration	SQL	Création et évolution du schéma de base
H2 Database	SQL	Base en mémoire pour le développement
Spring Boot Actuator	Ops	Point de santé `/actuator/health`
Spring Boot DevTools	Developer Tools	Redémarrage automatique en développement

Ne pas ajouter Lombok : le Java standard et les records suffisent, et Lombok affiche des avertissements sous Java 25.

Cliquer sur **GENERATE**, décompresser l'archive et ouvrir le dossier dans l'IDE. On obtient :

```
coffee-hexa-simple/
├── mvnw, mvnw.cmd, .mvn/        ← Maven Wrapper
├── pom.xml
└── src/
    ├── main/java/org/formation/coffee/CoffeeHexaSimpleApplication.java
    ├── main/resources/application.properties
    └── test/java/org/formation/coffee/CoffeeHexaSimpleApplicationTests.java
```

1.2 Vérifier le pom.xml généré

Ouvrir `pom.xml` et vérifier :

- le parent `spring-boot-starter-parent` en version **4.1.1** et la propriété `<java.version>25</java.version>` ;
- la présence des starters cochés : `spring-boot-starter-webmvc`, `spring-boot-starter-validation`, `spring-boot-starter-data-jpa`, `spring-boot-starter-flyway`, `h2`, `spring-boot-starter-actuator`, `spring-boot-devtools` ;

- la présence, pour chacun, du starter de test associé (`spring-boot-starter-webmvc-test`, `spring-boot-starter-data-jpa-test`…), ajouté automatiquement par Initializr ;
- la présence de `spring-boot-h2console`, qui fournit la console H2 dans Spring Boot 4. S'il manque, l'ajouter sans version, comme les autres starters.

Le parent gère les versions de toutes les bibliothèques Spring, d'Hibernate, Flyway, H2 et JSpecify : c'est pourquoi ces dépendances n'ont pas de `<version>`. Spring Boot 4 a des starters modulaires : chaque technologie a son propre starter, et chacun a son starter `-test` associé.

1.3 Ajouter MapStruct à la main

MapStruct n'est pas proposé par Initializr. C'est un *processeur d'annotations* : à la compilation, il lit les interfaces de mapping que vous écrivez et génère leur implémentation.

Dans `<properties>`, ajouter les versions :

```
<mapstruct.version>1.6.3</mapstruct.version>
<springdoc.version>3.1.0</springdoc.version>
```

Dans `<dependencies>`, ajouter MapStruct, ainsi que springdoc (Swagger UI), lui non plus absent d'Initializr :

```
<dependency>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct</artifactId>
    <version>${mapstruct.version}</version>
</dependency>
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>${springdoc.version}</version>
</dependency>
```

Dans `<build><plugins>`, à côté du `spring-boot-maven-plugin` déjà présent, déclarer le processeur d'annotations :

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <configuration>
        <annotationProcessorPaths>
            <path>
                <groupId>org.mapstruct</groupId>
                <artifactId>mapstruct-processor</artifactId>
                <version>${mapstruct.version}</version>
            </path>
        </annotationProcessorPaths>
        <compilerArgs>
            <arg>-Amapstruct.defaultComponentModel=spring</arg>
            <arg>-Amapstruct.defaultInjectionStrategy=constructor</arg>
            <arg>-Amapstruct.unmappedTargetPolicy=ERROR</arg>
        </compilerArgs>
    </configuration>
</plugin>
```

Les trois `compilerArgs` demandent à MapStruct :

- de faire de chaque mapper généré un bean Spring (`defaultComponentModel=spring`) ;
- d'utiliser l'injection par constructeur ;
- de **faire échouer la compilation** si un champ cible n'est pas mappé (`unmappedTargetPolicy=ERROR`) : impossible d'oublier un champ en silence.

> Si le `pom.xml` généré contient déjà un `maven-compiler-plugin` (par exemple avec un `annotationProcessorPaths` pour un autre processeur), ajouter le chemin MapStruct dans ce bloc existant au lieu d'en déclarer un second.

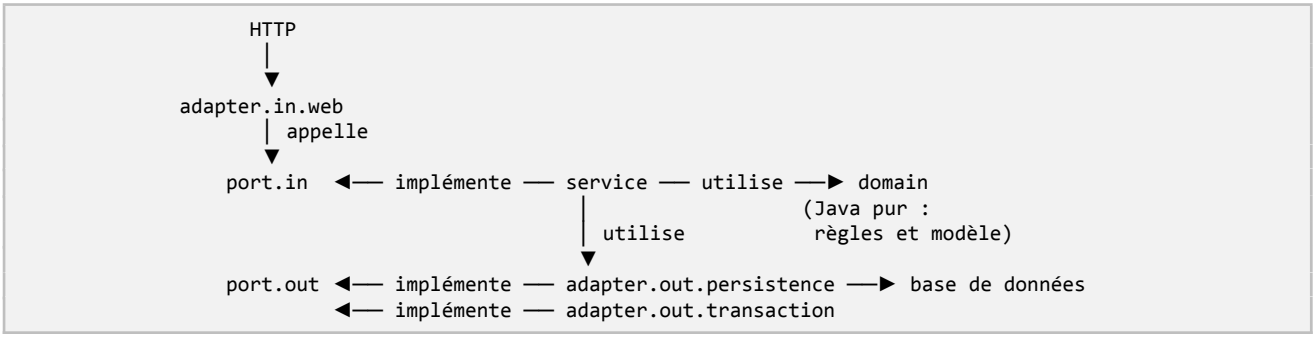
Recharger le projet Maven dans l'IDE.

Vérifier que le projet se construit. Il n'y a pas encore de code : cette commande télécharge seulement les dépendances.

```
./mvnw -q compile
```

Étape 2 — Comprendre l'architecture avant de coder

L'architecture hexagonale place **la logique métier au centre** et repousse **les détails techniques en périphérie** : HTTP, la base de données, le gestionnaire de transactions.



Couche	Package	Contient	Peut dépendre de
Domaine	`domain`	Modèle métier (`Coffee`, `Price`…) et règles métier	Rien (seulement les annotations JSpecify)
Application	`application`	Les **ports** (interfaces) et le **service** qui implémente les cas d'usage	Le domaine uniquement. Ni Spring, ni JPA.
Adaptateurs	`adapter.in.*`, `adapter.out.*`	Le code technique : contrôleur REST, JPA, transactions	Les ports de l'application et le domaine
Configuration	`config`	Le câblage Spring	Tout (elle assemble les pièces)

Le vocabulaire :

- **Port entrant** (`port.in`) : interface qui décrit ce que l'application *offre*, par exemple `CreateCoffeeUseCase`. Les adaptateurs entrants, comme le contrôleur REST, l'*appellent*.
- **Port sortant** (`port.out`) : interface qui décrit ce dont l'application *a besoin*, par exemple `LoadCoffeePort`. Les adaptateurs sortants, comme l'adaptateur de base de données, l'*implémentent*.
- **Adaptateur** : traduit entre le monde extérieur (JSON, SQL) et les ports.

Règle d'or : toutes les dépendances pointent vers l'intérieur, vers le domaine. Le domaine n'importe jamais Spring ni JPA. L'application n'importe jamais un adaptateur. C'est ce qui permet de tester la logique métier sans base de données, ou de remplacer H2 par PostgreSQL en ne touchant qu'un seul adaptateur.

On construit donc du centre vers l'extérieur : domaine → application → adaptateurs → câblage.

Étape 3 — La structure des packages

Voici l'arborescence complète visée à la fin du TP :

```
org.formation.coffee
├── CoffeeHexaSimpleApplication.java
├── domain
│   ├── model
│   │   ├── Coffee.java                (racine d'agrégat)
│   │   ├── CoffeeId.java              (identifiant typé)
│   │   ├── ReviewId.java              (identifiant typé)
│   │   ├── Price.java                 (objet-valeur)
│   │   └── Review.java
│   └── exception
│       ├── DomainException.java
│       ├── DomainValidationException.java
│       └── CoffeeNotFoundException.java
├── application
│   ├── port
│   │   ├── in                         (ce que l'application offre)
│   │   │   ├── ListCoffeesQuery.java
│   │   │   ├── GetCoffeeQuery.java
│   │   │   ├── CreateCoffeeUseCase.java
│   │   │   ├── UpdateCoffeeUseCase.java
│   │   │   ├── CreateCoffeeCommand.java
│   │   │   └── UpdateCoffeeCommand.java
│   │   └── out                        (ce dont l'application a besoin)
│   │       ├── LoadCoffeePort.java
│   │       ├── SaveCoffeePort.java
│   │       └── TransactionRunner.java
│   └── service
│       └── CoffeeApplicationService.java
├── adapter
│   ├── in
│   │   └── web
│   │       ├── CoffeeController.java
│   │       ├── CoffeeWebMapper.java
│   │       ├── RestExceptionHandler.java
│   │       └── dto
│   │           ├── CreateCoffeeRequest.java
│   │           ├── UpdateCoffeeRequest.java
│   │           ├── CoffeeResponse.java
│   │           └── ReviewResponse.java
│   └── out
│       ├── persistence
│       │   ├── CoffeeJpaEntity.java
│       │   ├── ReviewJpaEntity.java
│       │   ├── CoffeeJpaRepository.java
│       │   ├── CoffeePersistenceMapper.java
│       │   └── CoffeePersistenceAdapter.java
│       └── transaction
│           └── SpringTransactionRunner.java
└── config
    └── UseCaseConfig.java
```

Créer tous les dossiers d'un coup depuis la racine du projet (macOS, Linux ou Git Bash) :

```
cd src/main/java/org/formation/coffee
mkdir -p domain/model domain/exception \
         application/port/in application/port/out application/service \
         adapter/in/web/dto adapter/out/persistence adapter/out/transaction \
         config
cd -
```

Sous Windows sans Git Bash, créer les mêmes packages depuis l'IDE (clic droit → *New* → *Package*).

3.1 Null safety et package-info.java

Spring Boot 4 et Spring Framework 7 utilisent les annotations **JSpecify** pour indiquer ce qui peut être `null`. Poser `@NullMarked` sur un package signifie : *« dans ce package, rien n'est null sauf ce qui est marqué `@Nullable` »*. L'IDE signale alors les `null` passés par erreur.

Chaque package reçoit un fichier `package-info.java` portant cette annotation. Pour le package racine :

Fichier : `src/main/java/org/formation/coffee/package-info.java`

```
@NullMarked
package org.formation.coffee;

import org.jspecify.annotations.NullMarked;
```

Créer le même fichier dans chacun des packages suivants, en adaptant seulement la ligne `package` :

- `org.formation.coffee.domain.model`
- `org.formation.coffee.domain.exception`
- `org.formation.coffee.application.port.in`
- `org.formation.coffee.application.port.out`
- `org.formation.coffee.application.service`
- `org.formation.coffee.adapter.in.web`
- `org.formation.coffee.adapter.in.web.dto`
- `org.formation.coffee.adapter.out.persistence`
- `org.formation.coffee.adapter.out.transaction`
- `org.formation.coffee.config`

Exemple pour le premier :

Fichier : `src/main/java/org/formation/coffee/domain/model/package-info.java`

```
@NullMarked
package org.formation.coffee.domain.model;

import org.jspecify.annotations.NullMarked;
```

Étape 4 — Couche 1 : le domaine

Le domaine est du **Java pur** : ni Spring, ni JPA. Il porte le vocabulaire métier et les règles qui doivent toujours être vraies (les *invariants*).

4.1 Les exceptions du domaine

Quand une règle est violée, le domaine lève ses propres exceptions plutôt que des exceptions Java génériques. L'adaptateur web les transformera plus tard en erreurs HTTP.

Une classe de base commune permet d'attraper « toute erreur métier » à un seul endroit :

Fichier : `src/main/java/org/formation/coffee/domain/exception/DomainException.java`

```
package org.formation.coffee.domain.exception;

/**
 * Type de base de toutes les exceptions levées par le domaine.
 */
public abstract class DomainException extends RuntimeException {

    protected DomainException(String message) {
        super(message);
    }
}
```

Levée quand une valeur viole une règle (prix négatif, nom vide…) :

Fichier : `src/main/java/org/formation/coffee/domain/exception/DomainValidationException.java`

```
package org.formation.coffee.domain.exception;

/**
 * Levée quand une valeur ou une opération violerait un invariant métier.
 */
public class DomainValidationException extends DomainException {

    public DomainValidationException(String message) {
        super(message);
    }
}
```

Levée quand le café demandé n'existe pas. Elle conserve l'identifiant pour que la réponse d'erreur puisse l'inclure :

Fichier : `src/main/java/org/formation/coffee/domain/exception/CoffeeNotFoundException.java`

```
package org.formation.coffee.domain.exception;

import org.formation.coffee.domain.model.CoffeeId;

/**
 * Levée quand aucun café n'existe pour l'identifiant demandé.
 */
public class CoffeeNotFoundException extends DomainException {

    private final CoffeeId coffeeId;

    public CoffeeNotFoundException(CoffeeId coffeeId) {
        super("Coffee " + coffeeId.value() + " not found");
        this.coffeeId = coffeeId;
    }

    public CoffeeId getCoffeeId() {
        return coffeeId;
    }
}
```

4.2 Les objets-valeurs

Un **objet-valeur** enveloppe une valeur primitive et vérifie sa règle *une seule fois*, dans son constructeur. Un objet `Price` est donc toujours valide : il est impossible d'en créer un invalide.

Les **records** Java s'y prêtent bien. Le *constructeur compact* (`public Price { ... }`) s'exécute avant l'affectation des champs.

Fichier : `src/main/java/org/formation/coffee/domain/model/Price.java`

```
package org.formation.coffee.domain.model;

import org.formation.coffee.domain.exception.DomainValidationException;

/**
 * Prix d'un café, toujours compris entre {@link #MIN} et {@link #MAX} inclus.
 */
public record Price(int amount) {

    public static final int MIN = 0;
    public static final int MAX = 200;

    public Price {
        if (amount < MIN || amount > MAX) {
            throw new DomainValidationException(
                    "Price must be between %d and %d, got %d".formatted(MIN, MAX, amount));
        }
    }
}
```

Les **identifiants typés** évitent de confondre les identifiants. Avec un simple `Long`, rien n'empêche de passer l'id d'un avis là où on attend l'id d'un café. Avec `CoffeeId` et `ReviewId`, le compilateur détecte l'erreur.

Fichier : `src/main/java/org/formation/coffee/domain/model/CoffeeId.java`

```
package org.formation.coffee.domain.model;

import org.formation.coffee.domain.exception.DomainValidationException;

/**
 * Identité d'un {@link Coffee}.
 */
public record CoffeeId(long value) {

    public CoffeeId {
        if (value <= 0) {
            throw new DomainValidationException("Coffee id must be positive, got " + value);
        }
    }
}
```

Fichier : `src/main/java/org/formation/coffee/domain/model/ReviewId.java`

```
package org.formation.coffee.domain.model;

import org.formation.coffee.domain.exception.DomainValidationException;

/**
 * Identité d'un {@link Review}.
 */
public record ReviewId(long value) {

    public ReviewId {
        if (value <= 0) {
            throw new DomainValidationException("Review id must be positive, got " + value);
        }
    }
}
```

4.3 Le record Review

Un avis est immuable et vérifie ses propres règles (auteur non vide, note de 1 à 5) :

Fichier : `src/main/java/org/formation/coffee/domain/model/Review.java`

```
package org.formation.coffee.domain.model;

import org.formation.coffee.domain.exception.DomainValidationException;

import java.util.Objects;

/**
 * Avis d'un client sur un café. Fait partie de l'agrégat {@link Coffee}.
 */
public record Review(ReviewId id, String author, String comment, int rating) {

    public static final int MIN_RATING = 1;
    public static final int MAX_RATING = 5;

    public Review {
        Objects.requireNonNull(id, "id must not be null");
        Objects.requireNonNull(comment, "comment must not be null");
        if (author == null || author.isBlank()) {
            throw new DomainValidationException("Review author must not be blank");
        }
        if (rating < MIN_RATING || rating > MAX_RATING) {
            throw new DomainValidationException(
                    "Rating must be between %d and %d, got %d".formatted(MIN_RATING, MAX_RATING, rating));
        }
    }
}
```

4.4 L'agrégat Coffee

`Coffee` est une **racine d'agrégat** : l'objet principal, propriétaire de ses avis. À remarquer :

- **Un constructeur privé et deux méthodes de fabrique**, qui disent clairement si le café est nouveau ou chargé depuis le stockage :
  - `Coffee.create(name, price)` crée un café **nouveau**, pas encore enregistré, donc sans identifiant ;
  - `Coffee.reconstitute(id, name, price, reviews)` **reconstruit** un café chargé depuis le stockage.
- **Aucun setter.** Les modifications passent par des méthodes nommées d'après des actions métier, `rename(...)` et `changePrice(...)`, qui vérifient les règles.
- **La liste des avis est immuable** (`List.copyOf`) : le code extérieur ne peut pas la modifier.
- **`equals` et `hashCode` reposent sur l'identifiant**, la règle d'identité habituelle d'une entité.

Fichier : `src/main/java/org/formation/coffee/domain/model/Coffee.java`

```
package org.formation.coffee.domain.model;

import org.formation.coffee.domain.exception.DomainValidationException;
import org.jspecify.annotations.Nullable;

import java.util.List;
import java.util.Objects;

/**
 * Racine d'agrégat du catalogue. Ses avis sont toujours chargés avec elle.
 */
public final class Coffee {

    public static final int MAX_NAME_LENGTH = 255;

    private final @Nullable CoffeeId id;
    private String name;
    private Price price;
    private final List<Review> reviews;

    private Coffee(@Nullable CoffeeId id, String name, Price price, List<Review> reviews) {
        this.id = id;
        this.name = requireValidName(name);
        this.price = Objects.requireNonNull(price, "price must not be null");
        this.reviews = List.copyOf(reviews);
    }

    /**
     * Crée un café pas encore enregistré, donc sans identité.
     */
    public static Coffee create(String name, Price price) {
        return new Coffee(null, name, price, List.of());
    }

    /**
     * Reconstruit un café qui existe déjà dans le stockage.
     */
    public static Coffee reconstitute(CoffeeId id, String name, Price price, List<Review> reviews) {
        return new Coffee(Objects.requireNonNull(id, "id must not be null"), name, price, reviews);
    }

    public void rename(String newName) {
        this.name = requireValidName(newName);
    }

    public void changePrice(Price newPrice) {
        this.price = Objects.requireNonNull(newPrice, "price must not be null");
    }

    public boolean isNew() {
        return id == null;
    }
```

```
    public @Nullable CoffeeId getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Price getPrice() {
        return price;
    }

    public List<Review> getReviews() {
        return reviews;
    }

    private static String requireValidName(@Nullable String name) {
        if (name == null || name.isBlank()) {
            throw new DomainValidationException("Coffee name must not be blank");
        }
        if (name.length() > MAX_NAME_LENGTH) {
            throw new DomainValidationException(
                    "Coffee name must not exceed %d characters".formatted(MAX_NAME_LENGTH));
        }
        return name;
    }

    @Override
    public boolean equals(@Nullable Object other) {
        if (this == other) {
            return true;
        }
        return other instanceof Coffee coffee && id != null && id.equals(coffee.id);
    }

    @Override
    public int hashCode() {
        return id != null ? id.hashCode() : System.identityHashCode(this);
    }

    @Override
    public String toString() {
        return "Coffee[id=" + id + ", name=" + name + ", price=" + price + "]";
    }
}
```

> **Point de contrôle :** `./mvnw -q compile` doit réussir. Le domaine compile sans rien savoir de Spring.

Étape 5 — Couche 2 : l'application (ports et service)

5.1 Les ports entrants : une interface par cas d'usage

Chaque interface fait **une seule** chose (principe de *ségrégation des interfaces*), et son nom indique si elle lit ou écrit :

- `...Query` pour une lecture ;
- `...UseCase` pour une écriture.

Fichier : src/main/java/org/formation/coffee/application/port/in/ListCoffeesQuery.java

```
package org.formation.coffee.application.port.in;

import org.formation.coffee.domain.model.Coffee;

import java.util.List;

public interface ListCoffeesQuery {

    List<Coffee> listCoffees();
}
```

Fichier : src/main/java/org/formation/coffee/application/port/in/GetCoffeeQuery.java

```
package org.formation.coffee.application.port.in;

import org.formation.coffee.domain.exception.CoffeeNotFoundException;
import org.formation.coffee.domain.model.Coffee;
import org.formation.coffee.domain.model.CoffeeId;

public interface GetCoffeeQuery {

    /**
     * @throws CoffeeNotFoundException si aucun café n'a cet identifiant
     */
    Coffee getCoffee(CoffeeId id);
}
```

Fichier : src/main/java/org/formation/coffee/application/port/in/CreateCoffeeUseCase.java

```
package org.formation.coffee.application.port.in;

import org.formation.coffee.domain.model.Coffee;

public interface CreateCoffeeUseCase {

    Coffee createCoffee(CreateCoffeeCommand command);
}
```

Fichier : src/main/java/org/formation/coffee/application/port/in/UpdateCoffeeUseCase.java

```
package org.formation.coffee.application.port.in;

import org.formation.coffee.domain.exception.CoffeeNotFoundException;
import org.formation.coffee.domain.model.Coffee;

public interface UpdateCoffeeUseCase {

    /**
     * @throws CoffeeNotFoundException si aucun café n'a l'identifiant de la commande
     */
    Coffee updateCoffee(UpdateCoffeeCommand command);
}
```

5.2 Les commandes

Une **commande** regroupe en un seul objet les entrées d'un cas d'usage d'écriture. Elle utilise des objets-valeurs (`Price`, `CoffeeId`) : une commande ne peut donc pas être construite avec des données invalides.

Fichier : `src/main/java/org/formation/coffee/application/port/in/CreateCoffeeCommand.java`

```
package org.formation.coffee.application.port.in;

import org.formation.coffee.domain.model.Price;

import java.util.Objects;

public record CreateCoffeeCommand(String name, Price price) {

    public CreateCoffeeCommand {
        Objects.requireNonNull(name, "name must not be null");
        Objects.requireNonNull(price, "price must not be null");
    }
}
```

Pour une mise à jour partielle (HTTP PATCH), un champ `null` signifie « ne pas le modifier » : c'est pourquoi ces champs sont `@Nullable`.

Fichier : `src/main/java/org/formation/coffee/application/port/in/UpdateCoffeeCommand.java`

```
package org.formation.coffee.application.port.in;

import org.formation.coffee.domain.model.CoffeeId;
import org.formation.coffee.domain.model.Price;
import org.jspecify.annotations.Nullable;

import java.util.Objects;

/**
 * Mise à jour partielle d'un café : un champ {@code null} reste inchangé.
 */
public record UpdateCoffeeCommand(CoffeeId id, @Nullable String name, @Nullable Price price) {

    public UpdateCoffeeCommand {
        Objects.requireNonNull(id, "id must not be null");
    }
}
```

5.3 Les ports sortants : ce dont l'application a besoin

L'application doit charger et enregistrer des cafés, mais *comment* ne la concerne pas. Ces interfaces décrivent le besoin ; l'adaptateur de base de données y répondra.

Fichier : `src/main/java/org/formation/coffee/application/port/out/LoadCoffeePort.java`

```
package org.formation.coffee.application.port.out;

import org.formation.coffee.domain.model.Coffee;
import org.formation.coffee.domain.model.CoffeeId;

import java.util.List;
import java.util.Optional;

/**
 * Charge des agrégats {@link Coffee} complets, avis compris.
 */
public interface LoadCoffeePort {

    List<Coffee> findAll();

    Optional<Coffee> findById(CoffeeId id);
}
```

Fichier : src/main/java/org/formation/coffee/application/port/out/SaveCoffeePort.java

```
package org.formation.coffee.application.port.out;

import org.formation.coffee.domain.model.Coffee;

public interface SaveCoffeePort {

    /**
     * Enregistre un café nouveau ou existant et le renvoie tel qu'enregistré, avec son identité.
     */
    Coffee save(Coffee coffee);
}
```

Les transactions sont aussi un besoin. Plutôt que de poser le `@Transactional` de Spring sur le service (ce qui le lierait à Spring), le service demande un `TransactionRunner` :

Fichier : src/main/java/org/formation/coffee/application/port/out/TransactionRunner.java

```
package org.formation.coffee.application.port.out;

import java.util.function.Supplier;

/**
 * Exécute une unité de travail dans une transaction, sans annotation de framework dans la couche application.
 */
public interface TransactionRunner {

    <T> T execute(Supplier<T> work);

    <T> T executeReadOnly(Supplier<T> work);
}
```

5.4 Le service applicatif

Le service implémente les quatre ports entrants à l'aide des ports sortants. C'est une **classe Java ordinaire** : ni `@Service`, ni `@Transactional`. Elle sera déclarée à Spring à l'étape 8.

Lire attentivement `updateCoffee` : il charge le café (ou lève `CoffeeNotFoundException`), demande au domaine de le modifier (le domaine vérifie les règles), puis l'enregistre, le tout dans une seule transaction.

Fichier : src/main/java/org/formation/coffee/application/service/CoffeeApplicationService.java

```
package org.formation.coffee.application.service;

import org.formation.coffee.application.port.in.CreateCoffeeCommand;
import org.formation.coffee.application.port.in.CreateCoffeeUseCase;
import org.formation.coffee.application.port.in.GetCoffeeQuery;
import org.formation.coffee.application.port.in.ListCoffeesQuery;
import org.formation.coffee.application.port.in.UpdateCoffeeCommand;
import org.formation.coffee.application.port.in.UpdateCoffeeUseCase;
import org.formation.coffee.application.port.out.LoadCoffeePort;
import org.formation.coffee.application.port.out.SaveCoffeePort;
import org.formation.coffee.application.port.out.TransactionRunner;
import org.formation.coffee.domain.exception.CoffeeNotFoundException;
import org.formation.coffee.domain.model.Coffee;
import org.formation.coffee.domain.model.CoffeeId;
import org.formation.coffee.domain.model.Price;

import java.util.List;

/**
 * Implémente les cas d'usage des cafés. Java pur : câblé par {@code UseCaseConfig}, sans annotation Spring.
 */
public class CoffeeApplicationService
        implements ListCoffeesQuery, GetCoffeeQuery, CreateCoffeeUseCase, UpdateCoffeeUseCase {

    private final LoadCoffeePort loadCoffeePort;
    private final SaveCoffeePort saveCoffeePort;
    private final TransactionRunner transactionRunner;

    public CoffeeApplicationService(LoadCoffeePort loadCoffeePort,
                                    SaveCoffeePort saveCoffeePort,
                                    TransactionRunner transactionRunner) {
        this.loadCoffeePort = loadCoffeePort;
        this.saveCoffeePort = saveCoffeePort;
        this.transactionRunner = transactionRunner;
    }

    @Override
    public List<Coffee> listCoffees() {
        return transactionRunner.executeReadOnly(loadCoffeePort::findAll);
    }

    @Override
    public Coffee getCoffee(CoffeeId id) {
        return transactionRunner.executeReadOnly(() -> loadCoffee(id));
    }

    @Override
    public Coffee createCoffee(CreateCoffeeCommand command) {
        return transactionRunner.execute(
                () -> saveCoffeePort.save(Coffee.create(command.name(), command.price())));
    }
```

```
    @Override
    public Coffee updateCoffee(UpdateCoffeeCommand command) {
        return transactionRunner.execute(() -> {
            Coffee coffee = loadCoffee(command.id());
            String name = command.name();
            if (name != null) {
                coffee.rename(name);
            }
            Price price = command.price();
            if (price != null) {
                coffee.changePrice(price);
            }
            return saveCoffeePort.save(coffee);
        });
    }

    private Coffee loadCoffee(CoffeeId id) {
        return loadCoffeePort.findById(id).orElseThrow(() -> new CoffeeNotFoundException(id));
    }
}
```

> **Point de contrôle :** `./mvnw -q compile` réussit. Tout le cœur de l'application (domaine + application) compile sans le moindre import Spring. Pour le vérifier :

```
grep -r "springframework" src/main/java/org/formation/coffee/domain
src/main/java/org/formation/coffee/application
# (aucune sortie = c'est bon)
```

Étape 6 — Couche 3a : les adaptateurs sortants (base de données, transactions)

Place au côté technique. Dans un adaptateur, tout est **package-private** (sans mot-clé `public`) sauf ce qui doit absolument être public : seuls les ports sont destinés à être utilisés de l'extérieur. Spring sait trouver et injecter des classes package-private.

6.1 Les entités JPA

Les entités JPA associent des classes Java à des tables. Elles sont **volontairement séparées des classes du domaine** :

- la forme de la base peut évoluer sans toucher au code métier ;
- le domaine reste libre de toute annotation `@Entity`.

`@OrderBy("id")` renvoie les avis dans un ordre stable.

Fichier : src/main/java/org/formation/coffee/adapter/out/persistence/CoffeeJpaEntity.java

```
package org.formation.coffee.adapter.out.persistence;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToMany;
import jakarta.persistence.OrderBy;
import jakarta.persistence.Table;
import org.jspecify.annotations.Nullable;

import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "coffee")
class CoffeeJpaEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private @Nullable Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private int price;

    @OneToMany(mappedBy = "coffee", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("id")
    private List<ReviewJpaEntity> reviews = new ArrayList<>();

    protected CoffeeJpaEntity() {
        // pour JPA et MapStruct
    }

    public @Nullable Long getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getPrice() {
        return price;
    }
```

```
    public void setPrice(int price) {
        this.price = price;
    }

    public List<ReviewJpaEntity> getReviews() {
        return reviews;
    }
}
```

Fichier : `src/main/java/org/formation/coffee/adapter/out/persistence/ReviewJpaEntity.java`

```
package org.formation.coffee.adapter.out.persistence;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import org.jspecify.annotations.Nullable;

@Entity
@Table(name = "review")
class ReviewJpaEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private @Nullable Long id;

    @Column(nullable = false)
    private String author;

    @Column(nullable = false)
    private String comment;

    @Column(nullable = false)
    private int rating;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "coffee_id", nullable = false)
    private CoffeeJpaEntity coffee;

    protected ReviewJpaEntity() {
        // pour JPA
    }

    public @Nullable Long getId() {
        return id;
    }

    public String getAuthor() {
        return author;
    }

    public String getComment() {
        return comment;
    }

    public int getRating() {
        return rating;
    }
}
```

6.2 Le repository Spring Data

Spring Data écrit le SQL à partir des noms de méthodes. `@EntityGraph(attributePaths = "reviews")` demande à Hibernate de charger les cafés **et** leurs avis en **une seule requête SQL** (une jointure).

Sans cela, Hibernate exécuterait une requête supplémentaire par café au premier accès à ses avis. Ce bug de performance classique s'appelle le **problème N+1**.

Fichier : `src/main/java/org/formation/coffee/adapter/out/persistence/CoffeeJpaRepository.java`

```
package org.formation.coffee.adapter.out.persistence;

import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

interface CoffeeJpaRepository extends JpaRepository<CoffeeJpaEntity, Long> {

    /**
     * Charge tous les cafés avec leurs avis en une seule requête.
     */
    @EntityGraph(attributePaths = "reviews")
    List<CoffeeJpaEntity> findAllByOrderByIdAsc();

    @EntityGraph(attributePaths = "reviews")
    Optional<CoffeeJpaEntity> findWithReviewsById(Long id);
}
```

6.3 Le mapper de persistance (MapStruct)

Le mapper convertit les entités JPA en objets du domaine et inversement. MapStruct en génère l'essentiel :

- `toDomain` est écrite à la main (méthode `default`) car elle doit appeler la fabrique `Coffee.reconstitute(...)` ;
- les petites méthodes `default` (`toReviewId`, `toAmount`) apprennent à MapStruct à convertir les objets-valeurs ;
- `@Mapping(target = "...", ignore = true)` indique explicitement qu'un champ n'est pas copié : la base génère l'id, et les avis ne sont pas écrits par ce mapping.

Fichier : `src/main/java/org/formation/coffee/adapter/out/persistence/CoffeePersistenceMapper.java`

```
package org.formation.coffee.adapter.out.persistence;

import org.formation.coffee.domain.model.Coffee;
import org.formation.coffee.domain.model.CoffeeId;
import org.formation.coffee.domain.model.Price;
import org.formation.coffee.domain.model.Review;
import org.formation.coffee.domain.model.ReviewId;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import org.mapstruct.MappingTarget;

import java.util.List;
import java.util.Objects;

@Mapper
interface CoffeePersistenceMapper {

    default Coffee toDomain(CoffeeJpaEntity entity) {
        return Coffee.reconstitute(
                new CoffeeId(Objects.requireNonNull(entity.getId(), "a stored coffee has an id")),
                entity.getName(),
                new Price(entity.getPrice()),
                toDomainReviews(entity.getReviews()));
    }

    List<Review> toDomainReviews(List<ReviewJpaEntity> entities);

    Review toDomainReview(ReviewJpaEntity entity);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "reviews", ignore = true)
    CoffeeJpaEntity toNewEntity(Coffee coffee);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "reviews", ignore = true)
    void updateEntity(Coffee coffee, @MappingTarget CoffeeJpaEntity entity);

    default ReviewId toReviewId(Long id) {
        return new ReviewId(id);
    }

    default int toAmount(Price price) {
        return price.amount();
    }
}
```

6.4 L'adaptateur de persistance

L'adaptateur **implémente les ports sortants** à l'aide du repository et du mapper. Observer comment `save` traite les deux cas :

- **café nouveau** (`id == null`) : on l'insère ;
- **café existant** : on charge l'entité gérée et on y recopie les modifications. Hibernate écrit l'`UPDATE` au commit de la transaction.

Fichier : `src/main/java/org/formation/coffee/adapter/out/persistence/CoffeePersistenceAdapter.java`

```
package org.formation.coffee.adapter.out.persistence;

import org.formation.coffee.application.port.out.LoadCoffeePort;
import org.formation.coffee.application.port.out.SaveCoffeePort;
import org.formation.coffee.domain.exception.CoffeeNotFoundException;
import org.formation.coffee.domain.model.Coffee;
import org.formation.coffee.domain.model.CoffeeId;
import org.springframework.stereotype.Component;

import java.util.List;
import java.util.Optional;

@Component
class CoffeePersistenceAdapter implements LoadCoffeePort, SaveCoffeePort {

    private final CoffeeJpaRepository repository;
    private final CoffeePersistenceMapper mapper;

    CoffeePersistenceAdapter(CoffeeJpaRepository repository, CoffeePersistenceMapper mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }

    @Override
    public List<Coffee> findAll() {
        return repository.findAllByOrderByIdAsc().stream().map(mapper::toDomain).toList();
    }

    @Override
    public Optional<Coffee> findById(CoffeeId id) {
        return repository.findWithReviewsById(id.value()).map(mapper::toDomain);
    }

    @Override
    public Coffee save(Coffee coffee) {
        CoffeeId id = coffee.getId();
        if (id == null) {
            return mapper.toDomain(repository.save(mapper.toNewEntity(coffee)));
        }
        CoffeeJpaEntity managed = repository.findWithReviewsById(id.value())
                .orElseThrow(() -> new CoffeeNotFoundException(id));
        mapper.updateEntity(coffee, managed);
        return mapper.toDomain(managed);
    }
}
```

6.5 L'adaptateur de transactions

Il implémente le port `TransactionRunner` avec le `TransactionTemplate` de Spring, qui exécute du code dans une transaction de façon programmatique. La variante en lecture seule permet à la base et à Hibernate d'optimiser les lectures.

Fichier : `src/main/java/org/formation/coffee/adapter/out/transaction/SpringTransactionRunner.java`

```
package org.formation.coffee.adapter.out.transaction;

import org.formation.coffee.application.port.out.TransactionRunner;
import org.springframework.stereotype.Component;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.support.TransactionTemplate;

import java.util.function.Supplier;

@Component
class SpringTransactionRunner implements TransactionRunner {

    private final TransactionTemplate readWrite;
    private final TransactionTemplate readOnly;

    SpringTransactionRunner(PlatformTransactionManager transactionManager) {
        this.readWrite = new TransactionTemplate(transactionManager);
        this.readOnly = new TransactionTemplate(transactionManager);
        this.readOnly.setReadOnly(true);
    }

    @Override
    public <T> T execute(Supplier<T> work) {
        return readWrite.execute(status -> work.get());
    }

    @Override
    public <T> T executeReadOnly(Supplier<T> work) {
        return readOnly.execute(status -> work.get());
    }
}
```

> **Point de contrôle :** `./mvnw -q compile`. MapStruct génère `CoffeePersistenceMapperImpl` dans `target/generated-sources/annotations/`. Ouvrez ce fichier pour voir le code écrit à votre place.

Étape 7 — Couche 3b : l'adaptateur entrant (API REST)

7.1 Les DTO (Data Transfer Objects)

Les DTO sont les formes JSON de l'API. Ils sont séparés du domaine pour que le contrat d'API et le modèle métier puissent évoluer indépendamment. Leur nom indique leur sens : `...Request` entre, `...Response` sort.

Les DTO de requête ne vérifient que la **forme** de l'entrée (champ présent, non vide). Les **règles métier** (plage de prix, longueur du nom) restent dans le domaine : elles ne sont écrites qu'une fois.

Fichier : `src/main/java/org/formation/coffee/adapter/in/web/dto/CreateCoffeeRequest.java`

```
package org.formation.coffee.adapter.in.web.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

/**
 * Seule la forme de la requête est validée ici ; les règles métier (plage de prix, longueur du nom)
 * appartiennent au domaine.
 */
public record CreateCoffeeRequest(@NotBlank String name, @NotNull Integer price) {
}
```

Fichier : src/main/java/org/formation/coffee/adapter/in/web/dto/UpdateCoffeeRequest.java

```
package org.formation.coffee.adapter.in.web.dto;

import org.jspecify.annotations.Nullable;

/**
 * Mise à jour partielle : un champ absent reste inchangé.
 */
public record UpdateCoffeeRequest(@Nullable String name, @Nullable Integer price) {
}
```

Fichier : src/main/java/org/formation/coffee/adapter/in/web/dto/CoffeeResponse.java

```
package org.formation.coffee.adapter.in.web.dto;

import java.util.List;

public record CoffeeResponse(long id, String name, int price, List<ReviewResponse> reviews) {
}
```

Fichier : src/main/java/org/formation/coffee/adapter/in/web/dto/ReviewResponse.java

```
package org.formation.coffee.adapter.in.web.dto;

public record ReviewResponse(long id, String author, String comment, int rating) {
}
```

7.2 Le mapper web

Il convertit les DTO en commandes, et les objets du domaine en réponses. Comme il construit `new Price(amount)`, un prix invalide lève une `DomainValidationException` avant même l'appel du cas d'usage.

Fichier : `src/main/java/org/formation/coffee/adapter/in/web/CoffeeWebMapper.java`

```
package org.formation.coffee.adapter.in.web;

import org.formation.coffee.adapter.in.web.dto.CoffeeResponse;
import org.formation.coffee.adapter.in.web.dto.CreateCoffeeRequest;
import org.formation.coffee.adapter.in.web.dto.ReviewResponse;
import org.formation.coffee.adapter.in.web.dto.UpdateCoffeeRequest;
import org.formation.coffee.application.port.in.CreateCoffeeCommand;
import org.formation.coffee.application.port.in.UpdateCoffeeCommand;
import org.formation.coffee.domain.model.Coffee;
import org.formation.coffee.domain.model.CoffeeId;
import org.formation.coffee.domain.model.Price;
import org.formation.coffee.domain.model.Review;
import org.formation.coffee.domain.model.ReviewId;
import org.jspecify.annotations.Nullable;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

@Mapper
interface CoffeeWebMapper {

    CoffeeResponse toResponse(Coffee coffee);

    ReviewResponse toReviewResponse(Review review);

    CreateCoffeeCommand toCommand(CreateCoffeeRequest request);

    @Mapping(target = "id", source = "id")
    @Mapping(target = "name", source = "request.name")
    @Mapping(target = "price", source = "request.price")
    UpdateCoffeeCommand toCommand(long id, UpdateCoffeeRequest request);

    default long toValue(CoffeeId id) {
        return id.value();
    }

    default long toValue(ReviewId id) {
        return id.value();
    }

    default int toAmount(Price price) {
        return price.amount();
    }

    default CoffeeId toCoffeeId(long id) {
        return new CoffeeId(id);
    }

    default @Nullable Price toPrice(@Nullable Integer amount) {
        return amount == null ? null : new Price(amount);
    }
}
```

7.3 Le contrôleur

Le contrôleur dépend **uniquement des interfaces des ports entrants**, jamais de la classe du service. À remarquer :

- `@RequestMapping(path = "/api/coffees", version = "1")` utilise le **versionnement d'API** intégré à Spring Framework 7. Le client choisit la version avec un en-tête `API-Version` (configuré à l'étape 10).
- `@Valid` déclenche les vérifications du DTO (`@NotBlank`, `@NotNull`).
- `POST` renvoie **201 Created** avec un en-tête `Location` qui pointe vers le nouveau café.

- Ni `Optional`, ni `if (trouvé)` : quand un café manque, le cas d'usage lève une exception et le gestionnaire d'exceptions répond.

Fichier : `src/main/java/org/formation/coffee/adapter/in/web/CoffeeController.java`

```
package org.formation.coffee.adapter.in.web;

import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.formation.coffee.adapter.in.web.dto.CoffeeResponse;
import org.formation.coffee.adapter.in.web.dto.CreateCoffeeRequest;
import org.formation.coffee.adapter.in.web.dto.UpdateCoffeeRequest;
import org.formation.coffee.application.port.in.CreateCoffeeUseCase;
import org.formation.coffee.application.port.in.GetCoffeeQuery;
import org.formation.coffee.application.port.in.ListCoffeesQuery;
import org.formation.coffee.application.port.in.UpdateCoffeeUseCase;
import org.formation.coffee.domain.model.Coffee;
import org.formation.coffee.domain.model.CoffeeId;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.util.UriComponentsBuilder;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping(path = "/api/coffees", version = "1")
@Tag(name = "Coffees", description = "Coffee catalog")
class CoffeeController {

    private final ListCoffeesQuery listCoffeesQuery;
    private final GetCoffeeQuery getCoffeeQuery;
    private final CreateCoffeeUseCase createCoffeeUseCase;
    private final UpdateCoffeeUseCase updateCoffeeUseCase;
    private final CoffeeWebMapper mapper;

    CoffeeController(ListCoffeesQuery listCoffeesQuery,
                     GetCoffeeQuery getCoffeeQuery,
                     CreateCoffeeUseCase createCoffeeUseCase,
                     UpdateCoffeeUseCase updateCoffeeUseCase,
                     CoffeeWebMapper mapper) {
        this.listCoffeesQuery = listCoffeesQuery;
        this.getCoffeeQuery = getCoffeeQuery;
        this.createCoffeeUseCase = createCoffeeUseCase;
        this.updateCoffeeUseCase = updateCoffeeUseCase;
        this.mapper = mapper;
    }

    @GetMapping
    @Operation(summary = "List all coffees with their reviews")
    List<CoffeeResponse> listCoffees() {
        return listCoffeesQuery.listCoffees().stream().map(mapper::toResponse).toList();
    }
```

```
    @GetMapping("/{id}")
    @Operation(summary = "Get a coffee by its id")
    CoffeeResponse getCoffee(@PathVariable long id) {
        return mapper.toResponse(getCoffeeQuery.getCoffee(new CoffeeId(id)));
    }

    @PostMapping
    @Operation(summary = "Create a coffee")
    ResponseEntity<CoffeeResponse> createCoffee(@Valid @RequestBody CreateCoffeeRequest request,
                                                UriComponentsBuilder uriBuilder) {
        Coffee created = createCoffeeUseCase.createCoffee(mapper.toCommand(request));
        CoffeeResponse response = mapper.toResponse(created);
        URI location = uriBuilder.path("/api/coffees/{id}").buildAndExpand(response.id()).toUri();
        return ResponseEntity.created(location).body(response);
    }

    @PatchMapping("/{id}")
    @Operation(summary = "Partially update a coffee")
    CoffeeResponse updateCoffee(@PathVariable long id, @Valid @RequestBody UpdateCoffeeRequest request) {
        return mapper.toResponse(updateCoffeeUseCase.updateCoffee(mapper.toCommand(id, request)));
    }
}
```

7.4 Le gestionnaire d'exceptions

Il transforme les exceptions du domaine en réponses HTTP grâce à `ProblemDetail`, le type Spring du format d'erreur RFC 9457. Les erreurs levées par Spring lui-même (JSON invalide, échec de `@Valid`, version d'API inconnue) sont déjà prises en charge par Spring Boot une fois `spring.mvc.problemdetails.enabled=true` activé à l'étape 10.

Fichier : `src/main/java/org/formation/coffee/adapter/in/web/RestExceptionHandler.java`

```
package org.formation.coffee.adapter.in.web;

import org.formation.coffee.domain.exception.CoffeeNotFoundException;
import org.formation.coffee.domain.exception.DomainValidationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

/**
 * Traduit les exceptions du domaine en problem details RFC 9457.
 * Les exceptions du framework (validation, JSON invalide) sont déjà gérées par Spring Boot.
 */
@RestControllerAdvice
class RestExceptionHandler {

    @ExceptionHandler
    ProblemDetail handleNotFound(CoffeeNotFoundException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, exception.getMessage());
        problem.setTitle("Coffee not found");
        problem.setProperty("coffeeId", exception.getCoffeeId().value());
        return problem;
    }

    @ExceptionHandler
    ProblemDetail handleValidation(DomainValidationException exception) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                HttpStatus.UNPROCESSABLE_CONTENT, exception.getMessage());
        problem.setTitle("Business rule violated");
        return problem;
    }
}
```

Étape 8 — Brancher le tout

Le service applicatif n'a aucune annotation Spring : Spring ne le connaît donc pas encore. Une classe de configuration le déclare comme bean. Spring y injecte ensuite les adaptateurs, puisque ce sont des `@Component` qui implémentent les interfaces des ports.

`proxyBeanMethods = false` est le réglage recommandé quand les méthodes `@Bean` ne s'appellent pas entre elles : Spring démarre plus vite.

Fichier : `src/main/java/org/formation/coffee/config/UseCaseConfig.java`

```
package org.formation.coffee.config;

import org.formation.coffee.application.port.out.LoadCoffeePort;
import org.formation.coffee.application.port.out.SaveCoffeePort;
import org.formation.coffee.application.port.out.TransactionRunner;
import org.formation.coffee.application.service.CoffeeApplicationService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

/**
 * Branche les services applicatifs, écrits sans framework, dans le contexte Spring.
 */
@Configuration(proxyBeanMethods = false)
class UseCaseConfig {

    @Bean
    CoffeeApplicationService coffeeApplicationService(LoadCoffeePort loadCoffeePort,
                                                      SaveCoffeePort saveCoffeePort,
                                                      TransactionRunner transactionRunner) {
        return new CoffeeApplicationService(loadCoffeePort, saveCoffeePort, transactionRunner);
    }
}
```

C'est la **seule** classe qui connaît à la fois le service et le monde Spring. Le contrôleur demande un `CreateCoffeeUseCase`, et Spring lui fournit ce bean `CoffeeApplicationService`, puisqu'il implémente cette interface.

Vérifier que la classe principale générée par Initializr ressemble à ceci (seul le package compte) :

Fichier : `src/main/java/org/formation/coffee/CoffeeHexaSimpleApplication.java`

```
package org.formation.coffee;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CoffeeHexaSimpleApplication {

    public static void main(String[] args) {
        SpringApplication.run(CoffeeHexaSimpleApplication.class, args);
    }
}
```

Étape 9 — Le schéma de base avec Flyway

Flyway crée et fait évoluer le schéma de la base à partir de fichiers SQL versionnés. En production, on ne laisse jamais Hibernate générer les tables : le schéma doit être explicite, relu et versionné. On utilise Flyway

dès le premier jour, même quand le modèle change encore : la section 9.1 montre pourquoi cela ne coûte presque rien pendant la conception, et comment Hibernate peut quand même écrire un premier jet du SQL.

Créer les dossiers :

```
mkdir -p src/main/resources/db/migration src/main/resources/db/seed
```

Le schéma. Le format de nom `V<version>__<description>.sql` est obligatoire, avec **deux** tirets bas. Les contraintes `check` reprennent les règles métier comme dernière ligne de défense.

Fichier : `src/main/resources/db/migration/V1__create_coffee_schema.sql`

```
create table coffee
(
    id    bigint generated by default as identity primary key,
    name  varchar(255) not null,
    price integer      not null,
    constraint ck_coffee_price check (price between 0 and 200)
);

create table review
(
    id        bigint generated by default as identity primary key,
    author    varchar(255) not null,
    comment   varchar(255) not null,
    rating    integer      not null,
    coffee_id bigint       not null,
    constraint fk_review_coffee foreign key (coffee_id) references coffee (id),
    constraint ck_review_rating check (rating between 1 and 5)
);

create index ix_review_coffee_id on review (coffee_id);
```

Les données de démonstration. Le préfixe `R__` en fait une migration *répétable*, exécutée après les migrations versionnées. Elle est rangée dans un dossier à part que seul le profil `dev` charge (étape 10) : la production ne reçoit jamais de fausses données.

Fichier : `src/main/resources/db/seed/R__demo_data.sql`

```
insert into coffee (name, price)
values ('Granador', 10),
       ('Tres Puntas', 70),
       ('Estrellas', 4);

insert into review (author, comment, rating, coffee_id)
select 'Ana', 'Intense', 5, id from coffee where name = 'Granador';
insert into review (author, comment, rating, coffee_id)
select 'Beto', 'Equilibre', 4, id from coffee where name = 'Granador';
insert into review (author, comment, rating, coffee_id)
select 'Cleo', 'Trop fort', 2, id from coffee where name = 'Tres Puntas';
```

9.1 Faire évoluer le schéma pendant la conception

« Écrire le SQL à la main alors que le modèle change encore, c'est trop lourd. » En réalité, non :

- **Tant que le schéma n'a été déployé nulle part, on modifie librement `V1__create_coffee_schema.sql`.** La base de développement est en mémoire et reconstruite à chaque démarrage : rien ne garde la trace de l'ancienne version de `V1`. Changer une colonne revient à modifier l'entité, modifier `V1`, redémarrer.
- **La règle « ne jamais modifier une migration déjà appliquée » s'applique dès qu'une base qui conserve ses données l'a exécutée** (serveur de test partagé, recette, production). À partir de là, chaque changement est un nouveau fichier : `V2__add_origin_to_coffee.sql`, `V3__`….

- **Si on oublie de mettre V1 à jour, l'application le signale.** Avec `ddl-auto=validate`, elle refuse de démarrer quand une entité et le schéma divergent, par exemple `Schema-validation: missing column [origin] in table [coffee]`.

Laisser Hibernate écrire un premier jet du SQL. On lance l'application une fois en envoyant la génération du schéma dans un fichier plutôt que dans la base. Flyway et la validation sont désactivés pour ce lancement, pour qu'il démarre même quand les entités sont en avance sur V1 :

```
./mvnw spring-boot:run -Dspring-boot.run.arguments="--spring.flyway.enabled=false
--spring.jpa.hibernate.ddl-auto=none
--spring.jpa.properties.jakarta.persistence.schema-generation.scripts.action=create
--spring.jpa.properties.jakarta.persistence.schema-generation.scripts.create-target=target/schema.sql
--spring.jpa.properties.hibernate.hbm2ddl.delimiter=;
--spring.jpa.properties.hibernate.hbm2ddl.schema-generation.script.append=false"
```

Attendre `Started CoffeeHexaSimpleApplication`, arrêter avec `Ctrl+C`, puis ouvrir `target/schema.sql` :

```
    create table coffee (
        price integer not null,
        id bigint generated by default as identity,
        name varchar(255) not null,
        primary key (id)
    );

    create table review (
        rating integer not null,
        coffee_id bigint not null,
        id bigint generated by default as identity,
        author varchar(255) not null,
        comment varchar(255) not null,
        primary key (id)
    );

    alter table if exists review
       add constraint FKq0vtakn3s6pgmuyt3nyj7y5my
       foreign key (coffee_id)
       references coffee;
```

Comparer avec le V1 écrit plus haut. La version générée est un **brouillon**, pas la migration :

Absent ou incorrect dans le brouillon	Pourquoi c'est important
Pas de `check (price between 0 and 200)` ni de `check (rating between 1 and 5)`	Hibernate ne connaît pas les règles métier : elles vivent dans le domaine, pas sur les entités.
Pas d'index sur `review.coffee_id`	La jointure qui charge les avis parcourrait toute la table à mesure qu'elle grossit.
Nom généré `FKq0vtakn3s6pgmuyt3nyj7y5my`	Illisible dans les messages d'erreur ; nommer soi-même les contraintes (`fk_review_coffee`).
Colonnes dans l'ordre interne d'Hibernate	Esthétique, mais `id` en premier se lit plus facilement.

Reporter les parties utiles dans V1 (ou dans un nouveau V2 une fois le schéma déployé), puis corriger les points du tableau ci-dessus. On profite de la génération, et le schéma reste un fichier qu'on a lu et relu.

> `append=false` fait écraser `target/schema.sql` à chaque lancement ; par défaut, Hibernate ajouterait à la fin du fichier. Le fichier est dans `target/` : `./mvnw clean` le supprime et il n'est jamais commité.

Étape 10 — Configurer l'application

Spring Boot lit toujours `application.properties`, et `application-<profil>.properties` uniquement quand ce profil est actif.

Remplacer le contenu de `application.properties` par :

Fichier : `src/main/resources/application.properties`

```
spring.application.name=coffee-hexa-simple
# Profil utilisé quand aucun n'est indiqué (./mvnw spring-boot:run) ; la production démarre avec un profil
explicite
spring.profiles.default=dev

# Persistance : Flyway possède le schéma, Hibernate vérifie seulement le mapping
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false

# Web : erreurs RFC 9457 et versionnement d'API par en-tête (Spring Framework 7)
spring.mvc.problemdetails.enabled=true
spring.mvc.apiversion.use.header=API-Version
spring.mvc.apiversion.default=1

# Java 25 : traiter les requêtes sur des threads virtuels
spring.threads.virtual.enabled=true

# La documentation d'API s'active par profil
springdoc.api-docs.enabled=false
springdoc.swagger-ui.enabled=false
```

Propriété	Rôle
`spring.profiles.default=dev`	Sans profil indiqué, l'application tourne en `dev`. La production démarrerait avec `--spring.profiles.active=prod`.
`ddl-auto=validate`	Hibernate **vérifie** que ses entités correspondent au schéma Flyway et refuse de démarrer sinon. Il ne modifie jamais le schéma. Quand on modifie une entité, on met aussi la migration à jour (section 9.1).
`open-in-view=false`	La session de base reste fermée hors transaction : aucune requête paresseuse cachée pendant l'écriture du JSON. Spring Boot affiche un avertissement au démarrage si la propriété n'est pas renseignée.
`problemdetails.enabled=true`	Les erreurs propres à Spring utilisent aussi le format JSON RFC 9457.
`apiversion.use.header` / `default`	La version d'API est lue dans l'en-tête `API-Version`, avec `1` par défaut si l'en-tête est absent.
`threads.virtual.enabled=true`	Chaque requête est traitée sur un *thread virtuel* (Java 21+) : une concurrence légère et qui passe à l'échelle.
`springdoc.*.enabled=false`	La documentation d'API est désactivée par défaut (plus sûr) ; le profil `dev` l'active.

Créer le fichier du profil `dev` :

Fichier : `src/main/resources/application-dev.properties`

```
# Base en mémoire avec un nom fixe pour que la console H2 puisse s'y connecter
spring.datasource.url=jdbc:h2:mem:coffee
spring.h2.console.enabled=true

# Les données de démonstration ne sont chargées qu'en dev
spring.flyway.locations=classpath:db/migration,classpath:db/seed

# Traces SQL via le logger plutôt que spring.jpa.show-sql
logging.level.org.hibernate.SQL=debug
spring.jpa.properties.hibernate.format_sql=true

springdoc.api-docs.enabled=true
springdoc.swagger-ui.enabled=true
```

Enfin, vérifier le test généré par Initializr. Il démarre toute l'application, ce qui prouve que le câblage fonctionne :

Fichier : `src/test/java/org/formation/coffee/CoffeeHexaSimpleApplicationTests.java`

```
package org.formation.coffee;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class CoffeeHexaSimpleApplicationTests {

    @Test
    void contextLoads() {
    }
}
```

Étape 11 — Construire, démarrer et observer

11.1 Construire et tester

```
./mvnw clean test
```

Le résultat attendu est `BUILD SUCCESS`. Le test a démarré l'application complète : migrations Flyway, validation Hibernate et tous les beans.

11.2 Démarrer

```
./mvnw spring-boot:run
```

Attendre le message `Started CoffeeHexaSimpleApplication`. Les logs montrent aussi Flyway qui applique `1 - create coffee schema` puis `demo data`. On arrête l'application avec `Ctrl+C`.

11.3 Appeler l'API

Ouvrir un second terminal.

Lister les cafés. Observer les logs de l'application : une **seule** requête `select`, grâce à l'entity graph.

```
curl -s localhost:8080/api/coffees

[{"id":1,"name":"Granador","price":10,"reviews":[{"id":1,"author":"Ana","comment":"Intense","rating":5},{"id"
:2,"author":"Beto","comment":"Equilibre","rating":4}]},{"id":2,"name":"Tres
Puntas","price":70,"reviews":[{"id":3,"author":"Cleo","comment":"Trop
fort","rating":2}]},{"id":3,"name":"Estrellas","price":4,"reviews":[]}]
```

Obtenir un café :

```
curl -s localhost:8080/api/coffees/1
```

Café inconnu : problem detail 404.

```
curl -s -i localhost:8080/api/coffees/999

{"detail":"Coffee 999 not found","instance":"/api/coffees/999","status":404,"title":"Coffee not
found","coffeeId":999}
```

Créer un café : 201 et en-tête `Location`.

```
curl -s -i -X POST localhost:8080/api/coffees \
     -H 'Content-Type: application/json' \
     -d '{"name":"Yirgacheffe","price":25}'

HTTP/1.1 201
Location: http://localhost:8080/api/coffees/4
{"id":4,"name":"Yirgacheffe","price":25,"reviews":[]}
```

Violer une règle métier : 422, levée par l'objet-valeur `Price` du domaine.

```
curl -s -X POST localhost:8080/api/coffees \
     -H 'Content-Type: application/json' \
     -d '{"name":"Gold","price":500}'

{"detail":"Price must be between 0 and 200, got 500","instance":"/api/coffees","status":422,"title":"Business
rule violated"}
```

Requête mal formée : 400, levée par `@Valid` sur le DTO.

```
curl -s -X POST localhost:8080/api/coffees \
     -H 'Content-Type: application/json' \
     -d '{"name":" "}'
```

Mise à jour partielle : seul le prix change.

```
curl -s -X PATCH localhost:8080/api/coffees/1 \
     -H 'Content-Type: application/json' \
     -d '{"price":12}'
```

Versionnement d'API : la version 1 fonctionne, la version 2 est refusée.

```
curl -s -o /dev/null -w "%{http_code}\n" localhost:8080/api/coffees -H 'API-Version: 1'   # 200
curl -s localhost:8080/api/coffees -H 'API-Version: 2'                                    # 400, "Invalid
API version"
```

> **Windows PowerShell :** `curl` y est un alias d'une autre commande. Utiliser `curl.exe`, et mettre le JSON entre guillemets doubles en échappant les guillemets intérieurs, par exemple `-d "{\"name\":\"Gold\",\"price\":500}"`. On peut aussi passer par Swagger UI.

11.4 Outils dans le navigateur (profil dev uniquement)

Outil	URL	Remarques
Swagger UI	`http://localhost:8080/swagger-ui/index.html`	Essayer chaque endpoint depuis le navigateur
Console H2	`http://localhost:8080/h2-console`	JDBC URL `jdbc:h2:mem:coffee`, utilisateur `sa`, mot de passe vide
Santé	`http://localhost:8080/actuator/health`	`{"status":"UP"}`

Dans la console H2, exécuter `select * from coffee;` puis `select * from review;` après les appels précédents : on retrouve le café créé et le prix modifié.

Dépannage

Symptôme	Cause et solution
`invalid target release: 25`	Le terminal utilise un JDK plus ancien. Lancer `java -version` et faire pointer `JAVA_HOME` vers le JDK 25.
`Failed to parse configuration class [...]` qui cite une classe disparue	D'anciennes classes compilées restent dans `target/` après un déplacement ou un renommage. Lancer `./mvnw clean test`.
`Unmapped target property: "xyz"` à la compilation	Le mode strict de MapStruct fait son travail : un champ n'est pas mappé. Le mapper, ou ajouter `@Mapping(target = "xyz", ignore = true)` si c'est voulu.
L'IDE signale `CoffeeWebMapperImpl` / `CoffeePersistenceMapperImpl` introuvables	Activer le traitement des annotations (IntelliJ : *Settings → Build → Compiler → Annotation Processors → Enable*), ou lancer `./mvnw compile` une fois puis recharger le projet Maven.
`Schema-validation: missing table [coffee]`	Flyway ne s'est pas exécuté. Vérifier que le fichier s'appelle exactement `V1__create_coffee_schema.sql` (deux tirets bas) et se trouve dans `src/main/resources/db/migration`.
`Schema-validation: missing column [...]` ou `wrong column type`	Une entité a été modifiée mais pas la migration. Mettre `V1` à jour (ou ajouter un `V2` si le schéma est déjà déployé). Voir la section 9.1.
`Port 8080 was already in use`	Une autre application occupe le port. L'arrêter, ou ajouter `server.port=8081` dans `application-dev.properties`.
Avertissement `Using H2 2.4.x which is newer than the version Flyway has been verified with`	Sans conséquence en développement.

Validation

- `./mvnw clean test` affiche `BUILD SUCCESS`.
- `grep -r "springframework"` sur les packages `domain` et `application` ne renvoie rien.
- `GET /api/coffees` renvoie les trois cafés de démonstration avec leurs avis, en une seule requête SQL visible dans les logs.
- `POST /api/coffees` renvoie 201 avec un en-tête `Location`.
- Un prix de 500 renvoie 422, un nom vide renvoie 400, un café inconnu renvoie 404, au format problem details.
- `PATCH /api/coffees/1` avec `{"price":12}` ne modifie que le prix.
- L'en-tête `API-Version: 2` est refusé avec un 400.

Récapitulatif : les règles appliquées

1. **Les dépendances pointent vers l'intérieur.** Le domaine n'importe rien de technique ; l'application n'importe ni adaptateur ni Spring.
2. **Le domaine fait respecter les règles métier.** Les objets-valeurs (`Price`, `CoffeeId`) et les méthodes de l'agrégat (`rename`, `changePrice`) rendent les états invalides impossibles.
3. **Les ports sont des interfaces qui appartiennent à l'application** : entrants pour ce qu'elle offre, sortants pour ce dont elle a besoin, une interface par cas d'usage.
4. **Les adaptateurs traduisent, rien de plus** : JSON ↔ commandes, entités ↔ domaine, exceptions ↔ codes HTTP.
5. **Les adaptateurs cachent leurs détails** en restant package-private.
6. **Une seule classe de configuration** relie le cœur en Java pur à Spring.
7. **Le schéma de base est du code** (Flyway), et Hibernate se contente de le valider. On modifie `V1` librement tant que la base est en mémoire, et on ajoute `V2`, `V3`… dès qu'une vraie base l'a appliqué.
8. **Les choix par défaut de Spring Boot 4 / Framework 7** : null safety avec JSpecify, versionnement d'API natif, problem details, threads virtuels, `open-in-view=false`.

Questions

- Pourquoi le domaine n'importe-t-il ni Spring ni JPA ? Que gagne-t-on concrètement pour les tests ?
- Pourquoi le service applicatif n'a-t-il ni `@Service` ni `@Transactional` ? Qui ouvre et ferme les transactions ?
- Pourquoi garder deux classes distinctes `Coffee` et `CoffeeJpaEntity` au lieu d'annoter directement le domaine ?
- La règle « prix entre 0 et 200 » est vérifiée dans le domaine et dans une contrainte SQL, mais pas dans le DTO. Pourquoi ?
- Pourquoi un prix de 500 donne-t-il un 422 alors qu'un nom vide donne un 400 ?
- Que se passerait-il sans `@EntityGraph` sur le repository ? Comment l'observer dans les logs ?
- Quels fichiers faudrait-il modifier pour remplacer H2 par PostgreSQL ? Le domaine et l'application changent-ils ?
- On ajoute un champ `origin` à `Coffee`. Que faut-il modifier, et dans quel ordre, avant puis après le premier déploiement en recette ?

Pour aller plus loin

- **Tester sans base de données :** tester le domaine et le service avec JUnit seul et de faux ports (implémentations en mémoire de `LoadCoffeePort` et `SaveCoffeePort`).
- **Tester chaque adaptateur :** le contrôleur avec `@WebMvcTest` et `MockMvcTester`, l'adaptateur de persistance avec `@DataJpaTest`.
- **Faire respecter l'architecture :** écrire un test ArchUnit qui fait échouer le build si une classe du domaine importe Spring.

- **Verrouillage optimiste :** ajouter `@Version` à `CoffeeJpaEntity` pour que deux mises à jour simultanées ne s'écrasent pas en silence.